

Bookshop Project Guide — Learning Maven and Java Together

Companion guide for the bookshop learning project

August 2026

How to Use This Guide

This guide turns your `bookshop` project into a hands-on lab. The rule is simple: **every Maven concept you read about must touch this project before you move on.** Reading about a plugin does nothing; configuring it, running it, and breaking it on purpose is what makes it stick.

Each part below has two tracks:

- **Java track** — code you write in `src/`.
- **Maven track** — pom changes and terminal commands.

Do them in order. Each part ends with a checklist. Don't skip the "break it on purpose" steps — seeing the failure message is half the lesson.

Where Your Project Stands Today

What you have right now:

- A pom with coordinates (`com.example:bookshop:1.0-SNAPSHOT`), a `dependencyManagement` block importing the JUnit 5.11 BOM, Gson 2.11 as a dependency, and plugin versions locked in `pluginManagement`.
- `App.java` — still "Hello World".
- `AppTest.java` — a test that only asserts `true`.

That is normal for where you are in the guide. The early chapters *are* pom work. But from here on, the project itself needs to grow.

Two Bugs Already in Your Pom (Fix These First)

Exercise 0.1 — the typo'd, unused property. Your `<properties>` block declares `juint.version` (note the typo: *juint*). Worse, nothing uses it — the JUnit BOM version is hardcoded to `5.11.0` further down. Fix both:

1. Rename the property to `junit.version`.
2. In the `junit-bom` dependency, replace the hardcoded `5.11.0` with `${junit.version}`.
3. Run `mvn dependency:tree` before and after to prove nothing changed.

Lesson: properties are simple text substitution, and Maven never warns you about an unused or misspelled property. You have to be careful.

Exercise 0.2 — pluginManagement vs plugins. All your plugins live inside `<pluginManagement>`. That block only *declares* versions and default configuration — it does not add behavior by itself (the default lifecycle bindings still pick these plugins up, which is why your build works). You'll feel the difference in Part 3 when you add real plugin configuration and it has to go in an actual `<plugins>` block.

Lesson: `pluginManagement` is to plugins what `dependencyManagement` is to dependencies — a version/config catalog, not an activation.

Part 1 — Give the Project Something to Build (Java Track)

Maven is boring when there's nothing to compile, test, or package. Build a tiny but real domain. Target: about 60 lines of Java total.

Exercise 1.1 — the Book class. Create `src/main/java/com/example/Book.java`:

- Fields: `title` (String), `author` (String), `price` (double).
- A constructor, getters, and a useful `toString()`.
- Java learning point: make the fields `private final` and think about why immutability is a good default.

Exercise 1.2 — the Bookshop class. Create `src/main/java/com/example/Bookshop.java`:

- Holds a `List<Book>`.
- `addBook(Book b)` — adds a book.
- `totalValue()` — sums the prices (try a stream: `books.stream().mapToDouble(Book::getPrice).sum()`).
- `findByAuthor(String author)` — returns a filtered list.
- `toJson()` — uses **Gson** to serialize the book list. This finally justifies the Gson dependency sitting in your pom:

```
Gson gson = new GsonBuilder().setPrettyPrinting().create();
return gson.toJson(books);
```

Exercise 1.3 — wire up `App.main`. Replace “Hello World”: create a `Bookshop`, add two or three books, print `totalValue()` and `toJson()`.

Exercise 1.4 — run it through Maven, not the IDE.

```
mvn compile          # watch target/classes fill up
mvn exec:java         # runs com.example.App (already configured in your pom)
```

`exec:java` works for you even from `pluginManagement` because you’re invoking the goal explicitly by name — this is a preview of Part 3.

Checklist for Part 1

☐

`mvn compile` succeeds and `target/classes/com/example/` contains your three `.class` files.

☐

`mvn exec:java` prints your books as JSON.

☐

You can explain why Gson needed no `<scope>` but JUnit needs `<scope>test</scope>`.

Part 2 — Real Tests and the Lifecycle

Exercise 2.1 — a test that can actually fail. Replace the body of `AppTest` (or better, create `BookshopTest.java`):

- Test that `totalValue()` returns the right sum for two known books.
- Test that `findByAuthor` returns only matching books.
- Test that `toJson()` output contains a book’s title (`assertTrue(json.contains("..."))` is fine at this stage).
- Use `junit-jupiter-params` (already in your pom!) for one `@ParameterizedTest` — e.g. several price combinations.

Exercise 2.2 — watch Surefire work.

```
mvn test
```

Note the `Tests run: N` summary and look inside `target/surefire-reports/`.

Exercise 2.3 — break it on purpose. Make one assertion wrong. Run `mvn test` (build fails), then:

```
mvn test -Dmaven.test.skip=true      # skips compiling & running tests
mvn package -DskipTests              # compiles tests but doesn't run them
```

Fix the test afterwards. *Lesson: the `test` phase is a quality gate inside the lifecycle; `package` will not run while tests fail.*

Exercise 2.4 — trace the default lifecycle. Run `mvn package` and read the log top to bottom. Write down the `plugin:goal` pairs you see executing, in order (resources, compiler, resources again for tests, compiler for tests, surefire, jar). That list *is* the default lifecycle for jar packaging — you just watched it.

Checklist for Part 2

☐

At least 3 real tests, one parameterized.

☐

You've seen a red build and know two ways to skip tests.

☐

You can name the phases

`validate` → `compile` → `test` → `package` → `verify` → `install` → `deploy` and say which plugin runs at each of the middle ones.

Part 3 — Plugins & Goals (Your Current Chapter)

This is where you are in the PDF guide, and this project is ready for it.

Concept in one paragraph: Maven itself does almost nothing — every real action (compiling, testing, jarring) is a *goal* provided by a *plugin*. Goals run either because a lifecycle *phase* they're bound to runs (`mvn package`), or because you call them directly by name (`mvn exec:java`, `mvn dependency:tree`).

Exercise 3.1 — call goals directly.

```
mvn dependency:tree                # where does junit-jupiter-api's version
come from?
mvn dependency:analyze             # any used-but-undeclared deps?
mvn help:describe -Dplugin=surefire # list a plugin's goals
mvn help:effective-pom > effective.pom # see what your pom REALLY is
```

Open `effective.pom` and find: the JUnit version injected by your BOM, and the super-pom defaults you never wrote (like `sourceDirectory`).

Exercise 3.2 — the classic jar-manifest exercise.

```
mvn package
java -jar target/bookshop-1.0-SNAPSHOT.jar
```

The second command fails: “*no main manifest attribute*”. Fix it by configuring `maven-jar-plugin` — and note this goes in a real `<plugins>` block (inside `<build>`, next to your existing `<pluginManagement>`), while the *version* stays managed where it is:

```
<build>
  <pluginManagement> ... </pluginManagement>
  <plugins>
    <plugin>
      <artifactId>maven-jar-plugin</artifactId>
      <configuration>
        <archive>
          <manifest>
            <mainClass>com.example.App</mainClass>
          </manifest>
        </archive>
      </configuration>
    </plugin>
  </plugins>
</build>
```

Re-run `mvn package` and `java -jar ...` — it should print your bookshop. But it will still crash: **Gson isn’t on the classpath** of a plain jar. Sit with that error; it motivates Exercise 3.4.

Exercise 3.3 — bind a goal to a phase yourself. Add an execution that prints a message during the build, so you see *binding* explicitly:

```

<plugin>
  <groupId>org.codehaus.mojo</groupId>
  <artifactId>exec-maven-plugin</artifactId>
  <executions>
    <execution>
      <id>announce</id>
      <phase>validate</phase>
      <goals><goal>exec</goal></goals>
      <configuration>
        <executable>echo</executable>
        <arguments><argument>Building the bookshop...</argument></arguments>
      </configuration>
    </execution>
  </executions>
</plugin>

```

Run `mvn package` — your message appears first, because `validate` is the first phase. Change `<phase>` to `package` and watch it move to the end. Delete this once the idea clicks (it's noise in a real build).

Exercise 3.4 — a fat jar with the Shade plugin. Solve Exercise 3.2's classpath crash: add `maven-shade-plugin`, bind its `shade` goal to the `package` phase (that *is* its default binding — write it out anyway), rebuild, and now `java -jar` fully works because Gson's classes are bundled inside your jar. Compare the two jar sizes in `target/`.

Checklist for Part 3

☐

You can explain phase vs goal vs plugin in your own words.

☐

`java -jar target/bookshop-1.0-SNAPSHOT.jar` runs your app, Gson included.

☐

You know why the jar-plugin *configuration* went in `<plugins>` while its *version* stayed in `<pluginManagement>`.

☐

You've read your effective pom once.

Part 4 — Next Chapters, Mapped to This Project

Do these as you reach the matching chapter in your PDF guide.

Properties & resources. Add `src/main/resources/shop.properties` with `shop.name=My Bookshop`. Load it in `App` with `getClass().getResourceAsStream("/shop.properties")`. Then enable Maven resource *filtering* and put `version=${project.version}` in the file — check `target/classes/shop.properties` to see the substitution happen.

Profiles. Create a `dev` profile that sets a property (e.g. `shop.greeting`) differently from the default. Run `mvn help:active-profiles` and `mvn package -Pdev`. Combine with resource filtering so the packaged file differs per profile.

Install & repositories. Run `mvn install`, then find your artifact under `~/.m2/repository/com/example/bookshop/`. That's the local repo the dependency chapters kept talking about — your own project is now a dependency someone else could declare.

Multi-module (the big one). Split the project:

- Parent pom (`packaging=pom`) holding `dependencyManagement` + `pluginManagement` — your existing management blocks finally move to where the comments in your pom said they belong (“may be moved to parent pom”).
- `bookshop-core` module: `Book`, `Bookshop`, the tests.
- `bookshop-app` module: `App`, depends on `bookshop-core`, gets the shade plugin.

Build from the parent with `mvn package` and watch the reactor order the modules for you.

Java track alongside: as the modules split, grow the domain — an `Inventory` interface with an in-memory implementation (interfaces & polymorphism), `Optional<Book> findByTitle(...)` (no more nulls), custom exception `BookNotFoundException` (checked vs unchecked), and records: rewrite `Book` as a `record` and see how much code vanishes.

Command Cheat Sheet

Command	What it teaches
<code>mvn compile</code>	compile phase, <code>target/classes</code>
<code>mvn test</code>	Surefire, quality gate
<code>mvn package</code>	full default lifecycle, jar creation
<code>mvn clean install</code>	clean lifecycle + local repository
<code>mvn exec:java</code>	calling a goal directly
<code>mvn dependency:tree</code>	transitive dependencies, BOM versions
<code>mvn dependency:analyze</code>	declared vs used dependencies
<code>mvn help:effective-pom</code>	inheritance from the super-pom
<code>mvn help:describe -Dplugin=X</code>	discovering a plugin's goals
<code>mvn help:active-profiles</code>	profile activation
<code>mvn package -X</code>	debug: every goal→phase binding

The One Habit That Matters

After every chapter, ask: *“what command or pom change proves I understood this?”* — and do it here, in the bookshop. If you can’t think of one, the chapter didn’t stick yet. This project is small on purpose: it’s a lab bench, not a product. Break it freely; `git init` it first so you can always roll back.